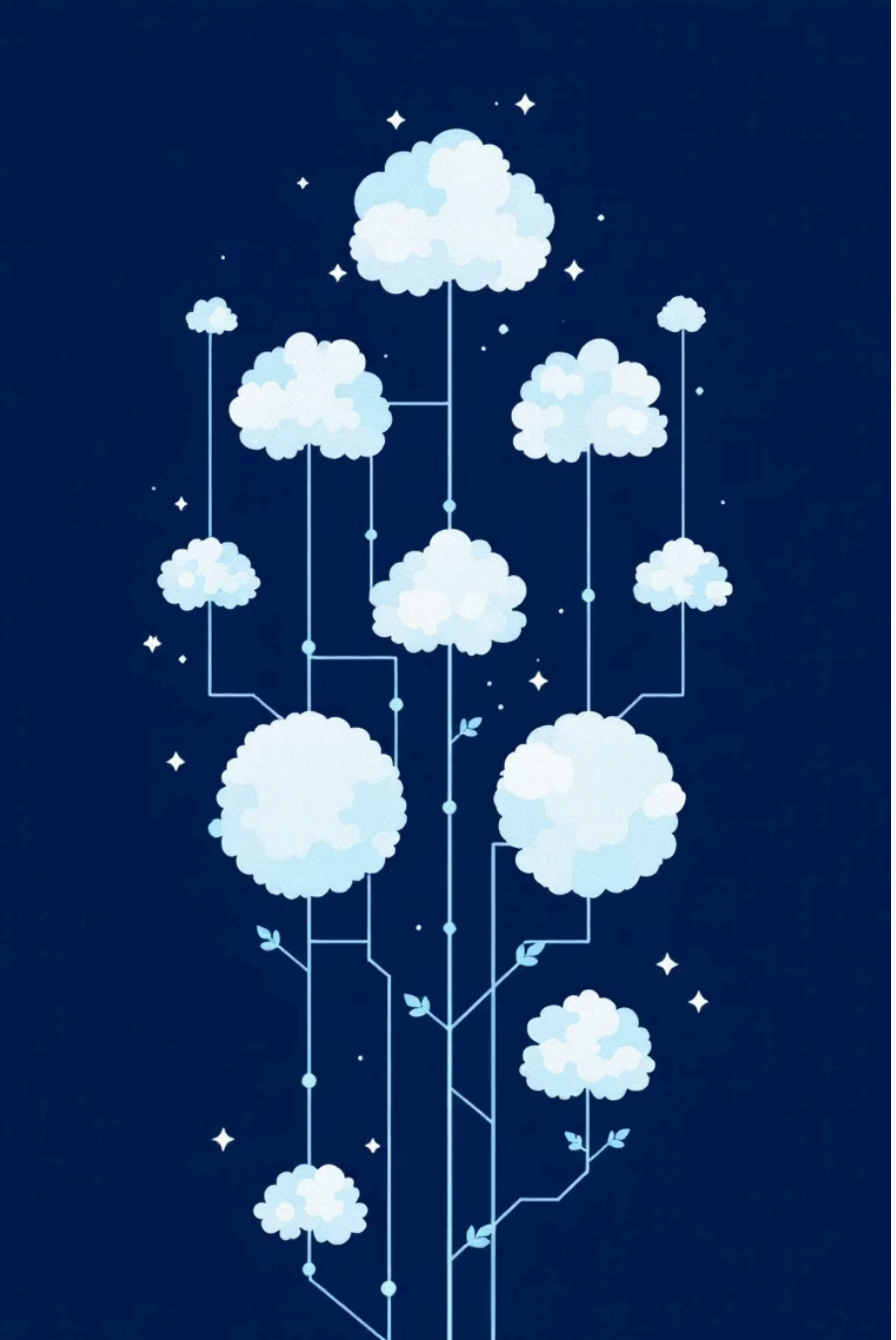


YBS 201 - Veri Yapıları ve Algoritmalar
İkili Arama Ağaçları (Binary Search Trees)

Doç. Dr. Ömer GÜLEÇ

www.pau.edu.tr/omer

Pamukkale Üniversitesi

Yönetim Bilişim Sistemleri



İkili Arama Ağaçları (Binary Search Trees)

Arama Uygulamaları

Arama işlemlerinin yoğun olduğu uygulamalarda ilk başvurulan model olarak öne çıkar. Veritabanı uygulamalarının can damarını oluşturur.

İkili Ağaçlardan Farkı

İkili ağaçlarda parent ile child verileri arasında büyüklük/küçüklük ilişkisi yoktur. BST'de ise bu ilişki kritik öneme sahiptir.

Sol Alt Ağaç İlişkisi

Her düğüm, sol alt ağacındaki tüm değerlerden büyüktür

Sağ Alt Ağaç İlişkisi

Her düğüm, sağ alt ağacındaki tüm değerlerden küçük ya da eşittir

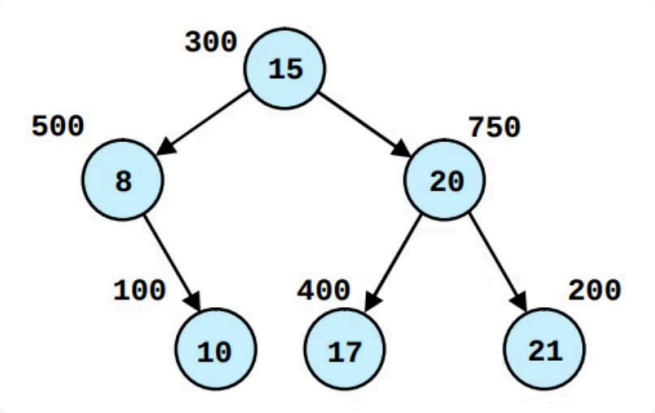
Özyinelemeli Yapı

Her düğümün alt ağacı yine bir BST'dir

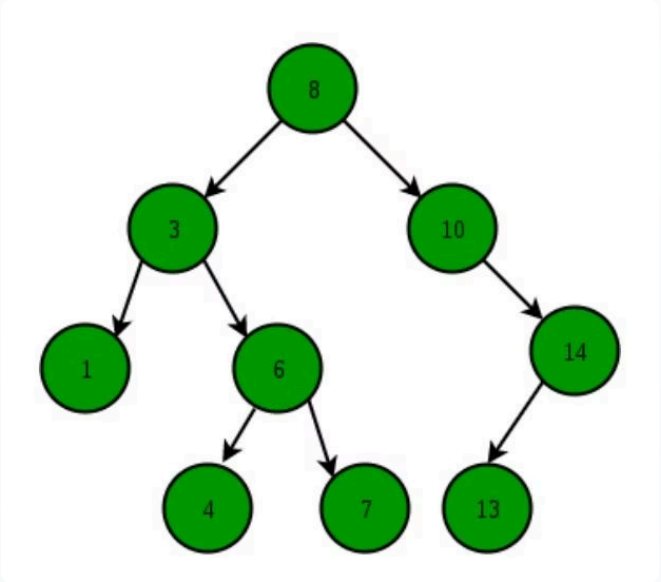
Sıralama Özelliği

Inorder sıralama yapıldığında küçükten büyüğe veriler elde edilir

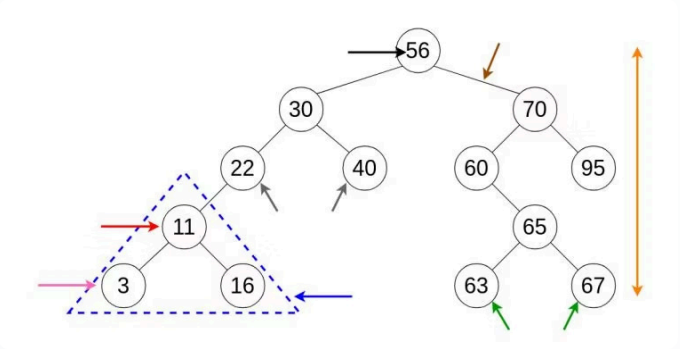
BST Yapısı ve Örnekleri



Temel BST yapısı: Her düğüm sol ve sağ çocuklarıyla büyüklük ilişkisi içindedir.



Dengeli BST örneği: Düğümler arasındaki hiyerarşik ilişki açıkça görülmektedir.



Karmaşık BST yapısı: Daha fazla düğüm içeren ağaç örneği.

İkili arama ağaçlarının temel yapısı, her düğümün sol tarafında kendinden küçük, sağ tarafında ise kendinden büyük veya eşit değerlerin bulunması prensibine dayanır. Bu yapı sayesinde arama işlemleri son derece verimli hale gelir.

NERELERDE KULLANILIR?

Binary Search Tree (BST) — yani **ikili arama ağacı**, bilgisayar bilimlerinde çok yaygın bir veri yapısıdır. Verileri **sıralı, hızlı erişilebilir ve düzenli** bir şekilde tutmak için kullanılır.

1. **Veri Arama ve Erişim İşlemleri**

- BST, verileri **sıralı** tuttuğu için, arama işlemleri ortalama durumda O(log n) zamanda yapılabilir.
- Bu yüzden:
  - **Telefon rehberi uygulamaları**
  - **Sözlük (dictionary) yapıları**
  - **Veri tabanı indeksleme** sistemlerinde sıkça kullanılır.

2. **Sıralama (Sorting) İşlemleri**

- Veriler bir BST'ye eklenip **in-order traversal** (kökün solundaki düğümler → kök → sağ düğümler) yapılarak **doğal sıralama** elde edilir.
  Böylece, **Tree Sort** algoritması oluşur.

3. **Veritabanı ve Dosya Sistemleri**

- **AVL Tree**, **Red-Black Tree**, **B-Tree**, **B+ Tree** gibi dengeli BST türevleri, veritabanlarında **indeksleme** ve **kayıt arama** işlemlerinde kullanılır.
  Örnek:
  - MySQL → B+ Tree tabanlı indeksleme
  - Linux dosya sistemleri (ext4, NTFS) → ağaç tabanlı dizin yapısı

4. **Bellek Yönetimi ve Derleyiciler**

- Derleyicilerde (compiler) **sembol tabloları** ve **değişken tanımları** BST ile tutulabilir.
- **Bellek tahsisi** ve **öncelikli kaynak yönetimi** için bazı sistemlerde ağaç yapıları tercih edilir.

5. **Oyun ve Yapay Zeka (AI) Uygulamaları**

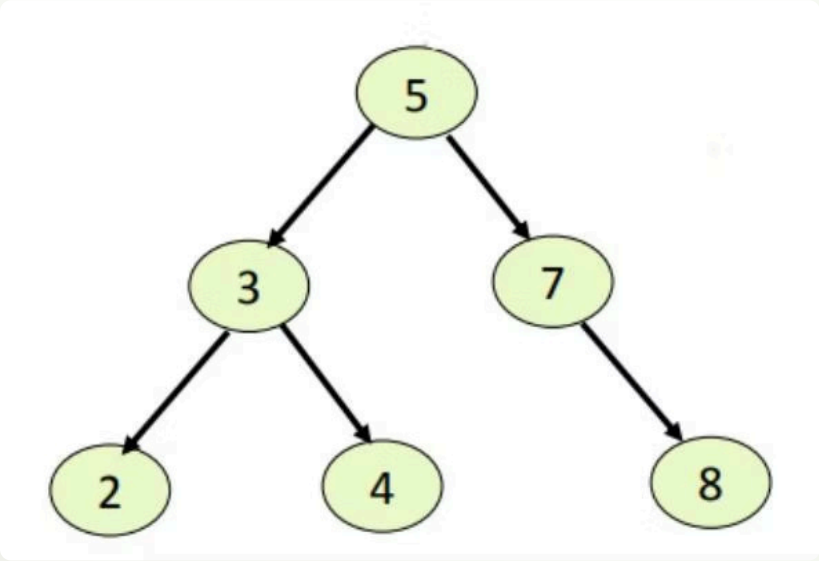
- **Karar ağaçları** (decision trees) ve **oyun durumlarının değerlendirilmesi** için BST benzeri yapılar (özellikle dengeli olanlar) kullanılır.

6. **Veri Sıkıştırma ve Şifreleme**

- Huffman kodlama gibi algoritmalarda **ağaç yapısı** kullanılır; bu da BST mantığına dayanır.
  - Örneğin: Huffman ağacı, sembolleri frekanslarına göre yerleştirir.

7. **Aralık ve Sıralı Sorgular**

- BST'nin "sıralı" doğası sayesinde şu tür sorgular kolaydır:
  - "10 ile 30 arasındaki tüm elemanları getir."
  - "Bir değerden küçük ya da büyük tüm düğümleri bul."
    Bu özellik, **coğrafi veri tabanları** ve **arama motorları** için kullanışlıdır.



BST Gezinme Yöntemleri

1

Preorder (Önce Kök)

Önce kök düğüm ziyaret edilir, ardından sol alt ağaç, son olarak sağ alt ağaç gezilir. Ağacın yapısını kopyalamak için idealdir.

2

Inorder (Ortada Kök)

Önce sol alt ağaç, sonra kök düğüm, en son sağ alt ağaç ziyaret edilir. BST'de küçükten büyüğe sıralı veri elde edilir.

3

Postorder (Sonda Kök)

Sol alt ağaç, sağ alt ağaç gezildikten sonra kök düğüm ziyaret edilir. Ağacı silmek için kullanılır.

Bu üç gezinme yöntemi, ağaç yapısındaki verilere farklı sıralarda erişim sağlar ve farklı algoritmik ihtiyaçlara cevap verir.

BST Oluşturma ve Ekleme İşlemi

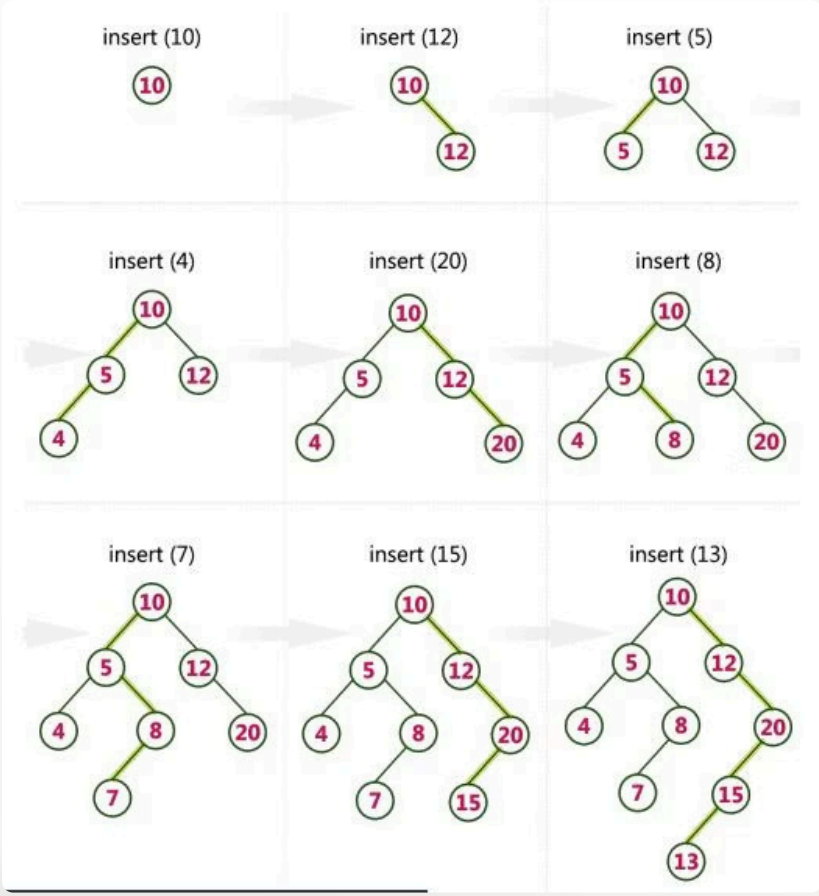
Oluşturma Süreci

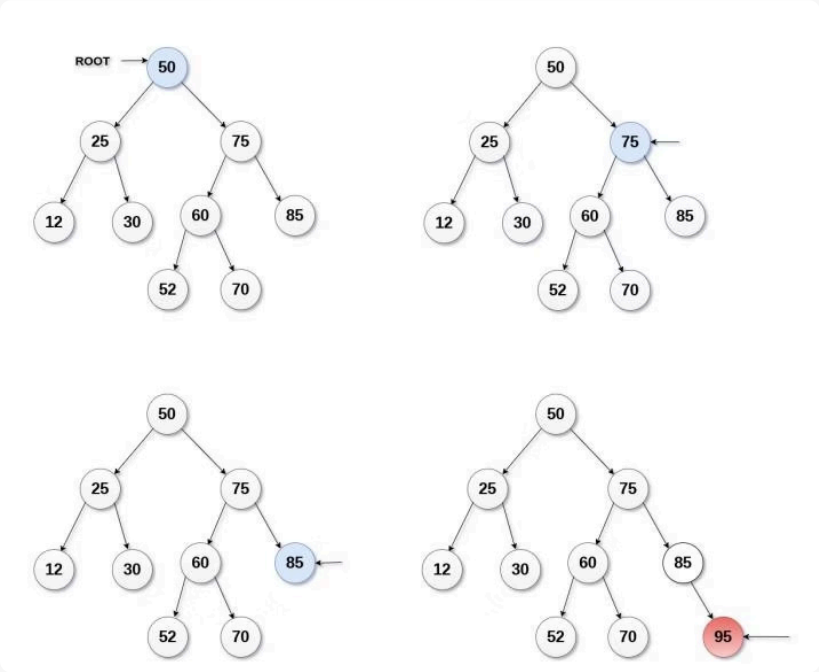
İkili arama ağacı oluşturulurken ilk eleman kök düğüm olarak yerleştirilir. Sonraki her eleman, kök düğümden başlayarak uygun konumuna yerleştirilir.

Ekleme işlemi sırasında, yeni değer mevcut düğümlerle karşılaştırılarak sol veya sağ alt ağaca yönlendirilir. Bu işlem, uygun boş konum bulunana kadar devam eder.

Ekleme Kuralları

- Eklenecek değer mevcut düğümden küçükse sol tarafa git
- Eklenecek değer mevcut düğümden büyük veya eşitse sağ tarafa git
- Boş konum bulunduğunda yeni düğümü oraya yerleştir
- Her ekleme işlemi BST özelliklerini korumalıdır





Ekleme İşlemi Detayları

Konum Arama

Kök düğümden başlayarak uygun pozisyon aranır

Karşılaştırma

Değer mevcut düğümle karşılaştırılır

Yönlendirme

Sol veya sağ alt ağaca yönlendirilir

Yerleştirme

Boş konuma yeni düğüm eklenir

Ekleme işlemi, ağacın dengeli kalmasını garanti etmez. Sıralı veri eklendiğinde ağaç dengesiz hale gelebilir ve performans düşebilir. Bu nedenle dengeli ağaç yapıları (AVL, Red-Black Tree) geliştirilmiştir.

Silme İşlemi:

Durum 1 - Yaprak Silme

Yaprak Düğüm

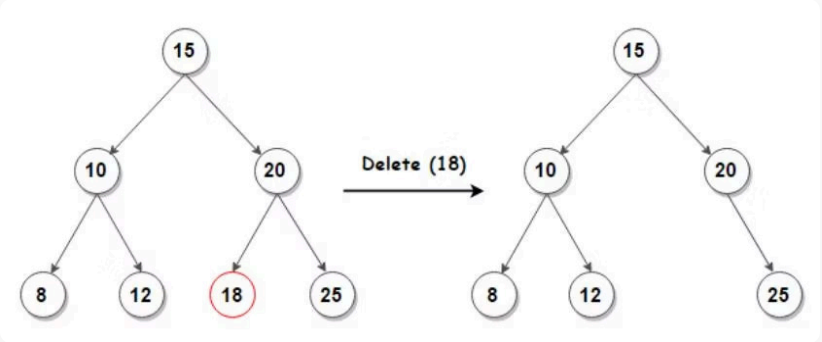
Silinecek düğümün hiç çocuğu yoktur. Bu, silme işleminin en basit durumudur.

Doğrudan Silme

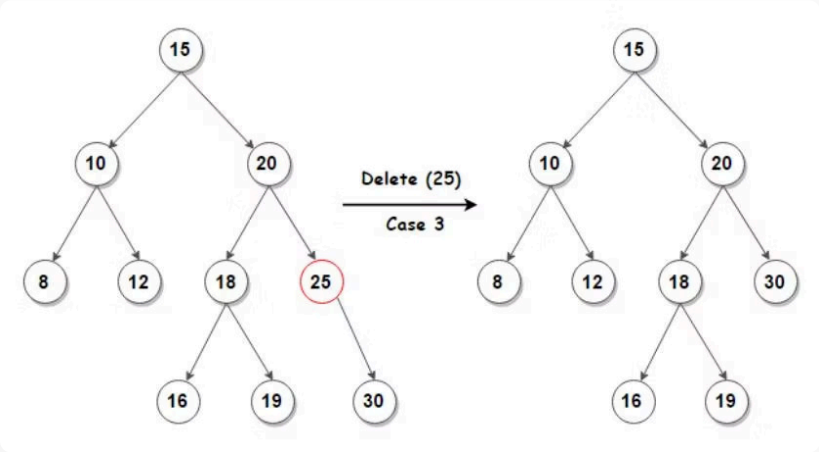
Düğüm doğrudan silinir ve parent düğümün ilgili pointer'ı NULL yapılır.

Ağaç Yapısı

Ağacın geri kalan yapısı etkilenmez, BST özellikleri korunur.



Yaprak düğüm silme işlemi, BST'deki en kolay silme senaryosudur. Düğümün çocuğu olmadığı için, sadece parent düğümün bağlantısını kesmek yeterlidir. Bu işlem ağacın dengesi veya diğer düğümlerin konumları üzerinde herhangi bir etki yaratmaz.



Silme İşlemi:
Durum 2 - Bir Çocuklu Düğüm

İşlem Adımları

1. Silinecek düğümün tek çocuğu belirlenir
2. Parent düğümün pointer'ı, silinecek düğümün çocuğuna yönlendirilir
3. Silinecek düğüm bellekten kaldırılır
4. Alt ağaç parent düğüme bağlanmış olur

Önemli Noktalar

Bir çocuklu düğüm silindiğinde, o düğümün çocuğu ve tüm alt ağacı, silinecek düğümün parent'ına bağlanır. Bu işlem BST özelliklerini korur çünkü alt ağaçtaki tüm değerler zaten doğru sıralamaya sahiptir.

Sol veya sağ çocuk olması fark etmez, işlem aynı mantıkla gerçekleştirilir.

Silme İşlemi:
Durum 3 - İki Çocuklu Düğüm

01

Yerine Geçecek Düğümü Bul

Sağ alt ağaçtaki minimum değer veya sol alt ağaçtaki maksimum değer bulunur

02

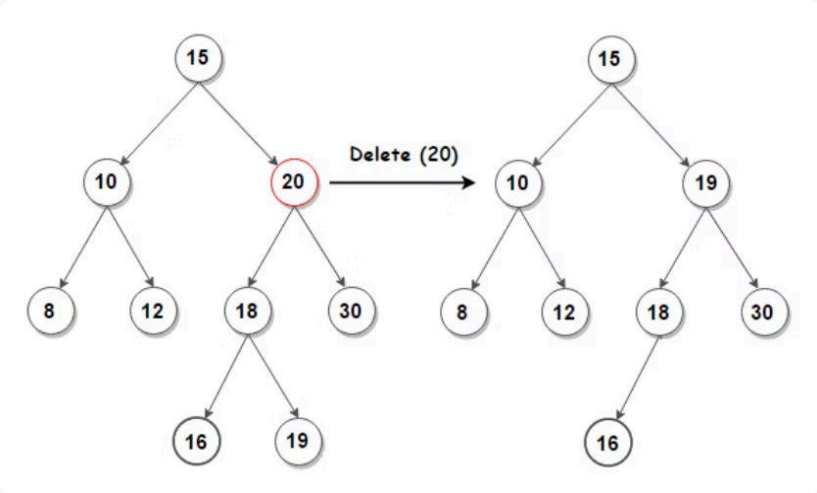
Değeri Kopyala

Bulunan düğümün değeri, silinecek düğüme kopyalanır

03

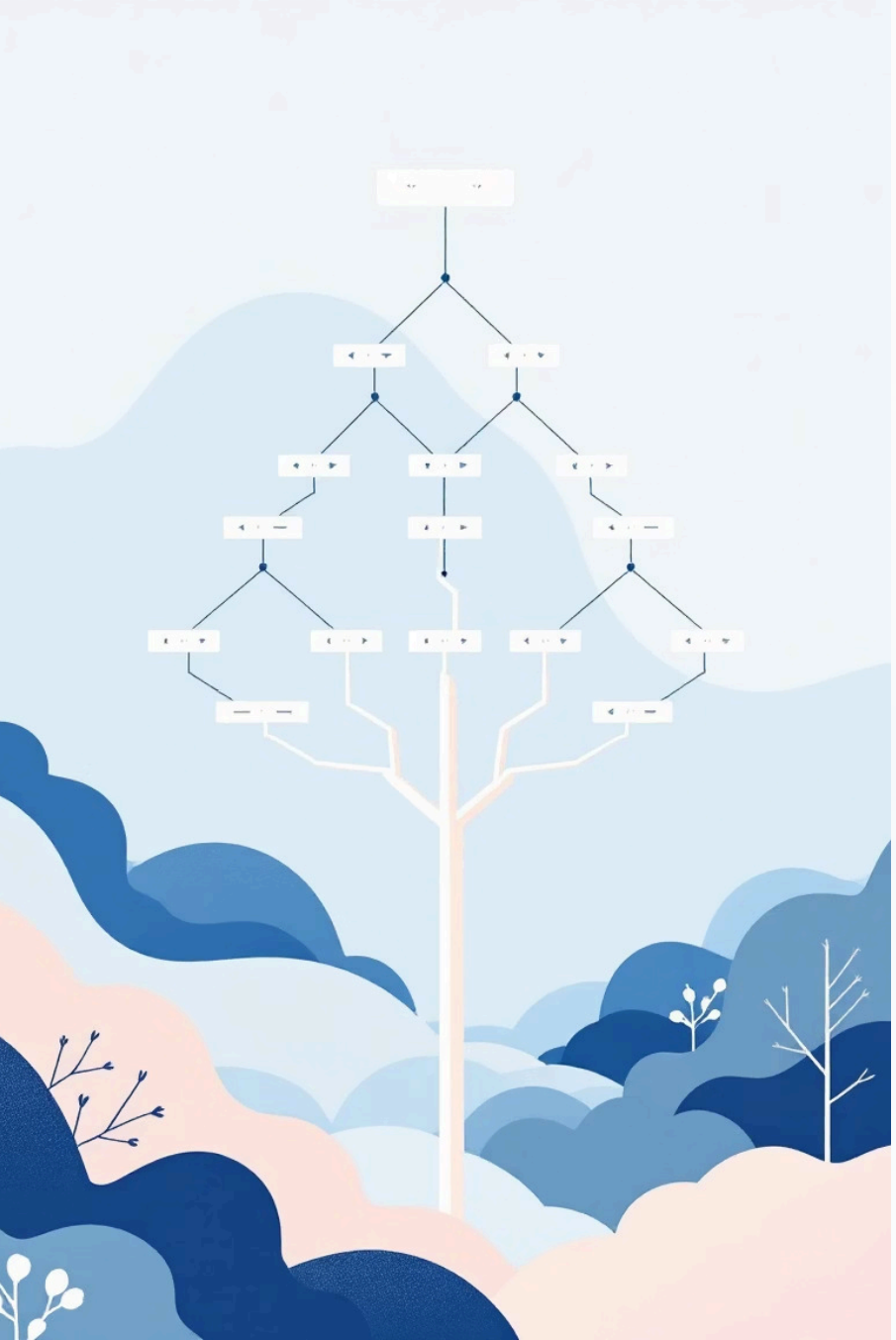
Yerine Geçen Düğümü Sil

Kopyalanan düğüm, Durum 1 veya Durum 2 kurallarıyla silinir



> **Kritik Nokta:** Sağ alt ağaçtaki minimum ya da sol alt ağaçtaki maksimum değer kullanılır. Bu değerler BST özelliklerini koruyarak silinecek düğümün yerini alabilir.

İki çocuklu düğüm silme, en karmaşık silme durumudur. Ancak doğru yerine geçecek düğüm seçildiğinde, BST'nin tüm özellikleri korunmuş olur ve ağaç yapısı bozulmaz.



AVL Ağaçları: Dengeli BST

Kendi Kendini Dengeleyen Yapı

Kendi kendini dengeleyen ikili arama ağacına AVL (Adelson-Velsky ve Landis) ağacı denir. Bu yapı, ağacın her zaman dengeli kalmasını garanti eder.

Denge Kriteri

Sağ alt ağaç ile sol alt ağaç arasındaki yükseklik farkının en fazla 1 düğüm olması gerekir. Bu kriter her düğüm için geçerlidir.

Otomatik Dengeleme

Normal BST'de ekleme ve çıkarma ile ağacın dengesi bozulur. AVL ile denge otomatik olarak sağlanır ve performans korunur.

AVL ağaçları, dengeli BST yapısıdır. Her ekleme ve silme işleminden sonra ağaç otomatik olarak dengelenir, böylece arama, ekleme ve silme işlemlerinin zaman karmaşıklığı O(log n) olarak garanti edilir.

AVL Ağaçları: Balans Faktörü

1

Sol Alt Ağaç Yüksekliği (hsol)

Sol alt ağacın kök düğümden itibaren en uzun yolunun uzunluğu

2

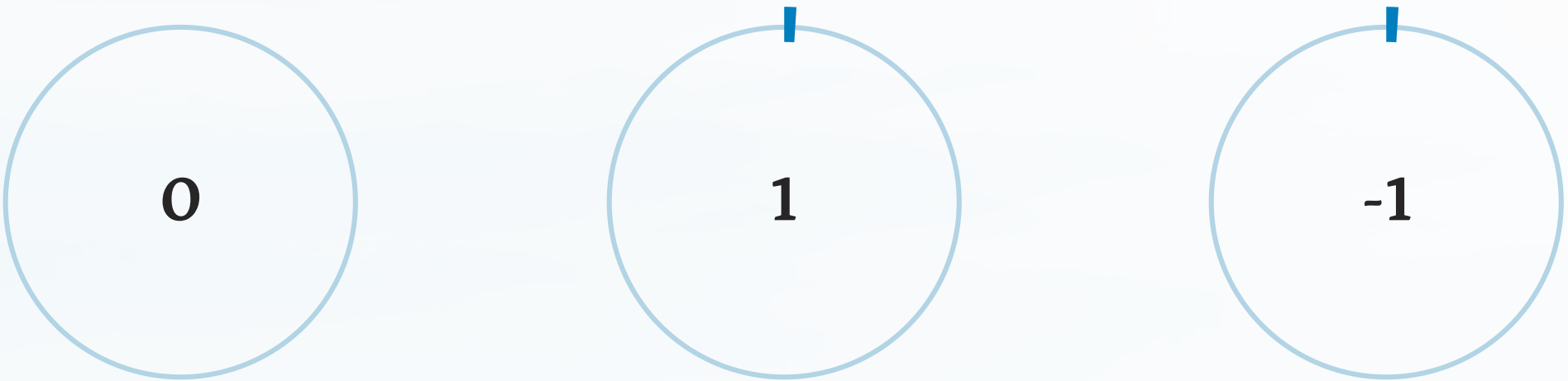
Sağ Alt Ağaç Yüksekliği (hsağ)

Sağ alt ağacın kök düğümden itibaren en uzun yolunun uzunluğu

3

Balans Faktörü Formülü

BF = hsol – hsağ şeklinde hesaplanır



Dengeli

Sol ve sağ alt ağaçlar eşit yükseklikte

Sol Fazla

Sol alt ağaç 1 seviye daha yüksek

Sağ Fazla

Sağ alt ağaç 1 seviye daha yüksek

Balans faktörü, AVL ağacının dengeli olup olmadığını belirleyen kritik ölçüttür. BF değeri 0, 1 veya -1 olabilir. Bu aralığın dışındaki değerler, ağacın dengesiz olduğunu ve döndürme işlemi gerektiğini gösterir.

AVL Ağaçları: Denge Örnekleri

Dengeli Ağaç Özellikleri

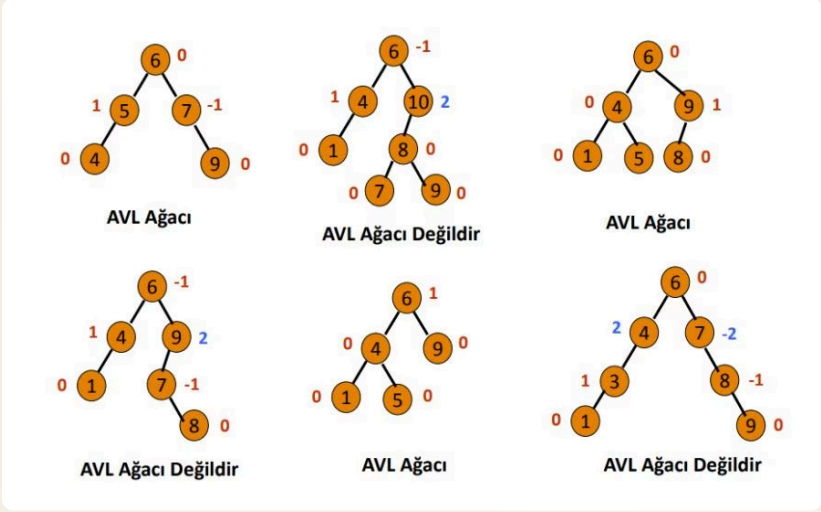
Yukarıdaki örnekte görüldüğü gibi, dengeli bir AVL ağacında her düğümün balans faktörü -1, 0 veya 1 değerlerinden birini alır.

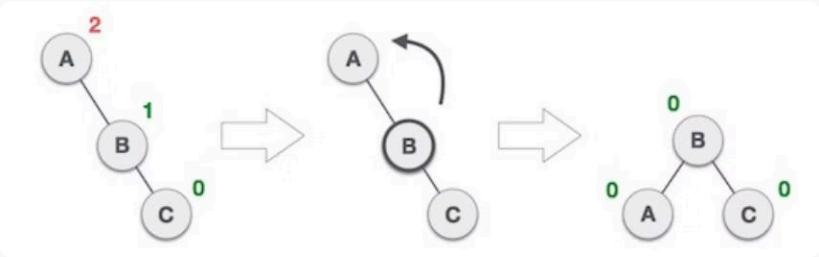
Bu denge sayesinde ağacın yüksekliği minimum seviyede tutulur ve tüm işlemler logaritmik zaman karmaşıklığında gerçekleştirilir.

Dengesiz Durumlar

Eğer herhangi bir düğümün balans faktörü 2 veya -2 olursa, ağaç dengesizdir ve döndürme işlemi gerekir.

Döndürme işlemleri (rotasyonlar) ağacı yeniden dengeler ve AVL özelliklerini geri kazandırır.





AVL Döndürme:
1 - Sola Döndürme

Dengesiz Durum

Sağ alt ağaç çok yüksek, BF = -2

Sola Döndürme

Sağ çocuk yukarı çıkar, parent aşağı iner

Dengeli Durum

Ağaç dengelenir, BF = 0 veya ±1

Sola döndürme (Left Rotation), sağ alt ağacın çok yüksek olduğu durumlarda kullanılır. Bu işlemde, dengesiz düğümün sağ çocuğu yeni kök olur ve eski kök sol çocuk konumuna geçer. Alt ağaçlar uygun şekilde yeniden düzenlenir ve BST özellikleri korunur.

Not: Sola döndürme işlemi O(1) zaman karmaşıklığına sahiptir ve sadece pointer değişiklikleri içerir.

AVL Döndürme:
2 - Sağa Döndürme

1

Sol Ağırlıklı Dengesizlik

Sol alt ağaç çok yüksek, balans faktörü +2 olur

2

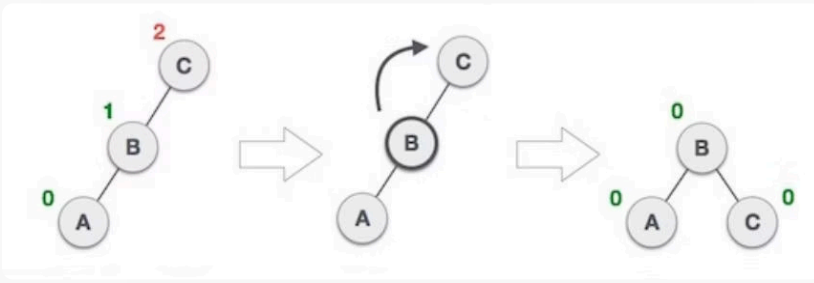
Sağa Döndürme Uygula

Sol çocuk yukarı çıkar ve yeni kök olur

3

Denge Sağlanır

Ağaç dengeli hale gelir, tüm BF değerleri ±1 aralığında



İşlem Detayları

Sağa döndürme (Right Rotation), sol alt ağacın çok yüksek olduğu durumlarda uygulanır. Dengesiz düğümün sol çocuğu yeni kök konumuna yükselir.

Eski kök, yeni kökün sağ çocuğu olur. Yeni kökün sağ alt ağacı varsa, eski köke sol çocuk olarak bağlanır.

Uygulama Senaryosu

Bu döndürme, özellikle azalan sırada veri eklendiğinde sık karşılaşılan bir durumdur.

Tek bir sağa döndürme işlemi ile ağaç dengelenir ve performans korunur.



AVL Döndürme:
3 - Sol ve Sağa Döndürme

Sol-Sağ Dengesizlik Tespiti

Sol çocuğun sağ alt ağacı yüksek olduğunda bu durum ortaya çıkar (Left-Right case)

Önce Sola Döndürme

Sol çocuk üzerinde sola döndürme işlemi uygulanır, bu durumu basit sol ağırlıklı duruma çevirir

Sonra Sağa Döndürme

Ana düğüm üzerinde sağa döndürme işlemi uygulanır ve ağaç dengelenir

Sol ve sağa döndürme (Left-Right Rotation), çift döndürme gerektiren karmaşık bir dengesizlik durumudur. Bu durum, sol alt ağacın sağ tarafının çok yüksek olması durumunda ortaya çıkar. İki aşamalı döndürme işlemi ile ağaç dengeli hale getirilir.

AVL Döndürme:
4 - Sağ ve Sola Döndürme

1. **Sağ-Sol Dengesizlik**

   Sağ çocuğun sol alt ağacı yüksek (Right-Left case)

2. **Sağa Döndür**

   Sağ çocuk üzerinde sağa döndürme uygula

3. **Sola Döndür**

   Ana düğüm üzerinde sola döndürme uygula

4. **Denge Sağlandı**

   Ağaç dengeli hale geldi



> **Önemli:** Sağ ve sola döndürme (Right-Left Rotation), sol ve sağa döndürmenin ayna görüntüsüdür. Sağ alt ağacın sol tarafının yüksek olduğu durumlarda kullanılır.

Bu çift döndürme işlemi, önce sağ çocuk üzerinde sağa döndürme, ardından ana düğüm üzerinde sola döndürme şeklinde gerçekleştirilir. İki aşamalı bu işlem sonunda ağaç dengeli hale gelir ve AVL özellikleri korunur.

AVL Örneği: [1, 2, 3, 4, 5, 6, 7]

Ekleme Süreci

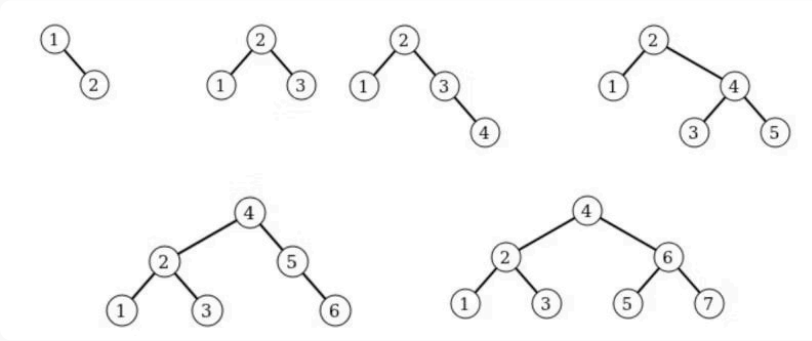
Sıralı sayılar (1, 2, 3, 4, 5, 6, 7) eklendiğinde, normal BST tamamen dengesiz bir yapı oluştururdu. Ancak AVL ağacında her ekleme sonrası otomatik dengeleme gerçekleşir.

Her yeni düğüm eklendiğinde balans faktörleri kontrol edilir ve gerekirse döndürme işlemleri uygulanır.

Sonuç Yapısı

Yukarıdaki diyagramda görüldüğü gibi, sıralı veri eklenmiş olmasına rağmen ağaç dengeli bir yapıya sahiptir.

Ağacın yüksekliği minimum seviyede tutulmuş ve tüm işlemler O(log n) karmaşıklığında gerçekleştirilebilir.



7

Toplam Düğüm

Ağaçta bulunan eleman sayısı

3

Ağaç Yüksekliği

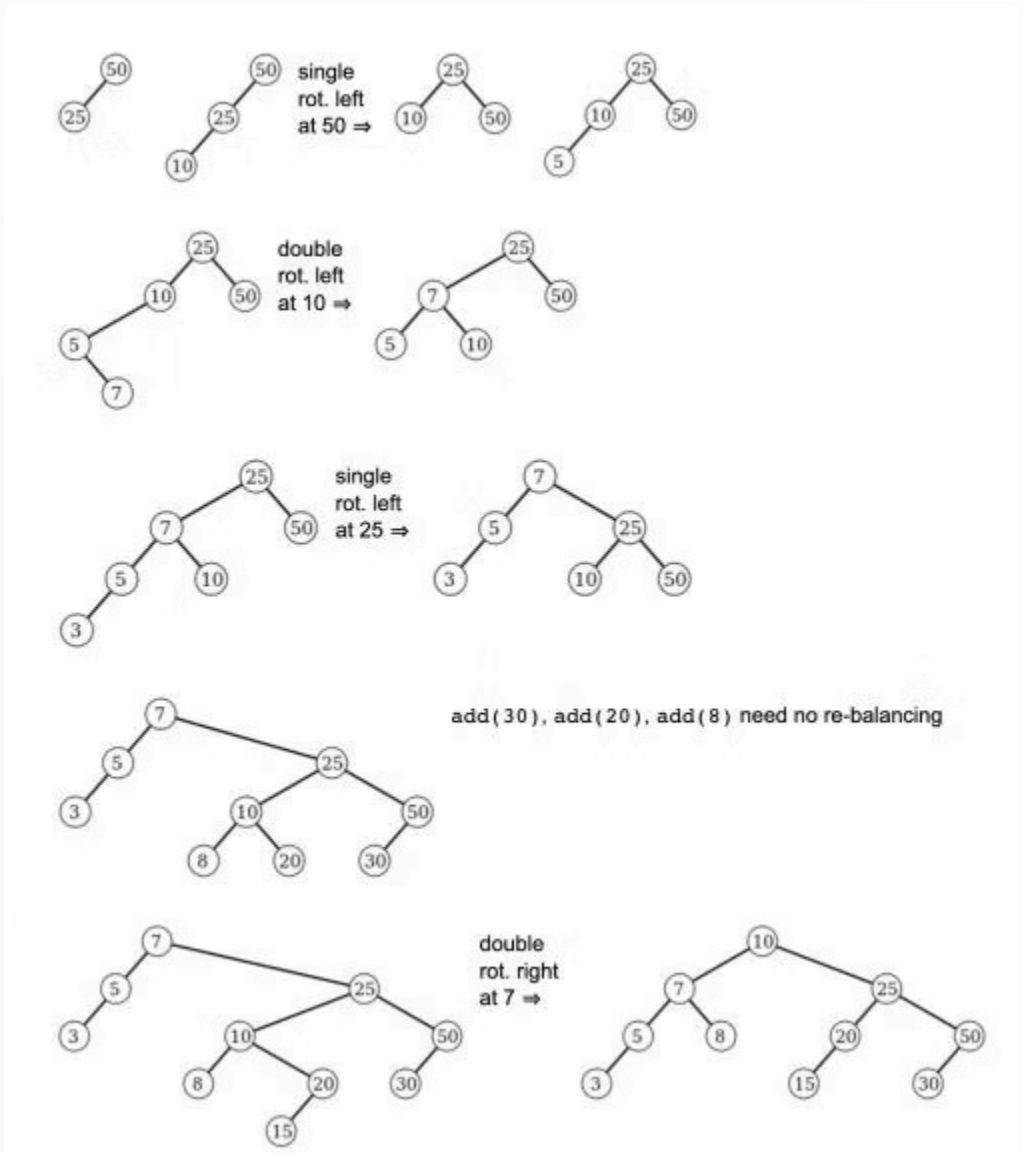
Dengeli yapı sayesinde minimum yükseklik

4

Kök Düğüm

Dengeleme sonrası ortadaki değer kök oldu

AVL Örneği: nodes = [50, 25, 10, 5, 7, 3, 30, 20, 8, 15]



Bu karmaşık örnekte, rastgele sırada eklenen değerler (50, 25, 10, 5, 7, 3, 30, 20, 8, 15) ile AVL ağacının nasıl dengeli kaldığı gösterilmektedir. Her ekleme işlemi sonrasında gerekli döndürme işlemleri otomatik olarak uygulanmış ve ağaç dengeli yapısını korumuştur.

Çoklu Döndürme İşlemleri

Bu örnek, birden fazla döndürme işleminin nasıl uygulandığını gösterir. Bazı eklemelerde tek döndürme, bazılarında çift döndürme gerekmiştir.

Dengeli Sonuç

10 elemanlı bu ağaç, dengeli yapısı sayesinde maksimum 4 seviye yüksekliğe sahiptir. Normal BST'de bu değerler çok daha dengesiz bir yapı oluşturabilirdi.

Performans Garantisi

AVL ağacı, her durumda O(log n) arama, ekleme ve silme performansı garanti eder. Bu örnekte 10 eleman için maksimum 4 karşılaştırma yeterlidir.

AVL ağaçları, veri yapıları arasında en dengeli ve performanslı yapılardan biridir. Otomatik dengeleme mekanizması sayesinde, en kötü durumda bile logaritmik zaman karmaşıklığı garanti edilir.